Desarrollo de Aplicaciones Móviles

Examen Parcial II

AgroSierra · AgroAPI y AgroCampo

Estudiante: Grease Duran Castro

Caso: Versión A · Grupo A

Fecha: 27 de octubre de 2026

Plataforma evaluada: Android

Backend: Kotlin, Spring Boot y H2 en archivo

Aplicación: Kotlin Multiplatform, Compose, Ktor, Koin y SQLDelight

Resultado del trabajo

Se construyen dos proyectos nuevos para administrar lotes y labores de campo. AgroAPI valida reglas y privilegios. AgroCampo consulta su base local, conserva cambios sin cobertura y sincroniza al volver la conexión. Se incluyen SC-A, SC-B, SC-C y SC-D.

Verificación	Resultado observado
Backend	9 pruebas de integración aprobadas
Android commonTest	13 pruebas aprobadas
Postman / Newman	27 solicitudes y 30 comprobaciones, sin fallos
Modo avión	Registro local -1, reapertura y envío automático como ID 23

Backend: github.com/GreysiLL/ExamenS2
Aplicación: github.com/GreysiLL/exa-agrocampo

Inicio real: 27 de octubre. No se atribuye a este caso un historial anterior ni se alteran fechas. La bitácora está en America/Lima; el reloj del emulador usa otra zona horaria.

1. Arquitectura y contrato

El servicio separa seguridad, usuarios, dominio, repositorios JDBC, servicios transaccionales y controladores. La aplicación separa dominio, DTO, persistencia, casos de uso, presentación con ViewModel/StateFlow e inyección con Koin.

Lectura: AgroAPI → DTO paginado → dominio → SQLDelight → Flow → UiState → Compose. La pantalla principal no toma sus listas de la respuesta HTTP.

Escritura: formulario → caso de uso → transacción local y cola → REST → confirmación local → descarga del catálogo.

Método y ruta bajo /api/v1	Contrato
POST /auth/login	Público; JWT firmado por una hora
GET /usuarios/me	Datos y privilegios del usuario
GET /lotes	Paginación, valle y estado
GET /lotes/{id}	Detalle o 404
GET /lotes/{id}/labores	Paginación; fecha descendente
POST /labores	201 y fila definitiva
PUT /labores/{id}	200 y labor corregida
DELETE /labores/{id}	204 sin cuerpo
POST /lotes/{id}/cerrar-campania	200; exige privilegio y cosecha

Listados: contenido, pagina, tamanio, totalElementos, totalPaginas y ultima. Errores: timestamp, status, error, message, path y validationErrors. El responsable se toma del usuario autenticado.

H2 persiste en data/agrosierra.mv.db y permite levantar el backend con un comando sin instalar otra base. Android usa SQLite mediante SQLDelight. Las pruebas HTTP se aíslan en otra base; la demostración usa el mismo servicio compilado con un archivo H2 de pruebas. No hay servicios simulados en producción.

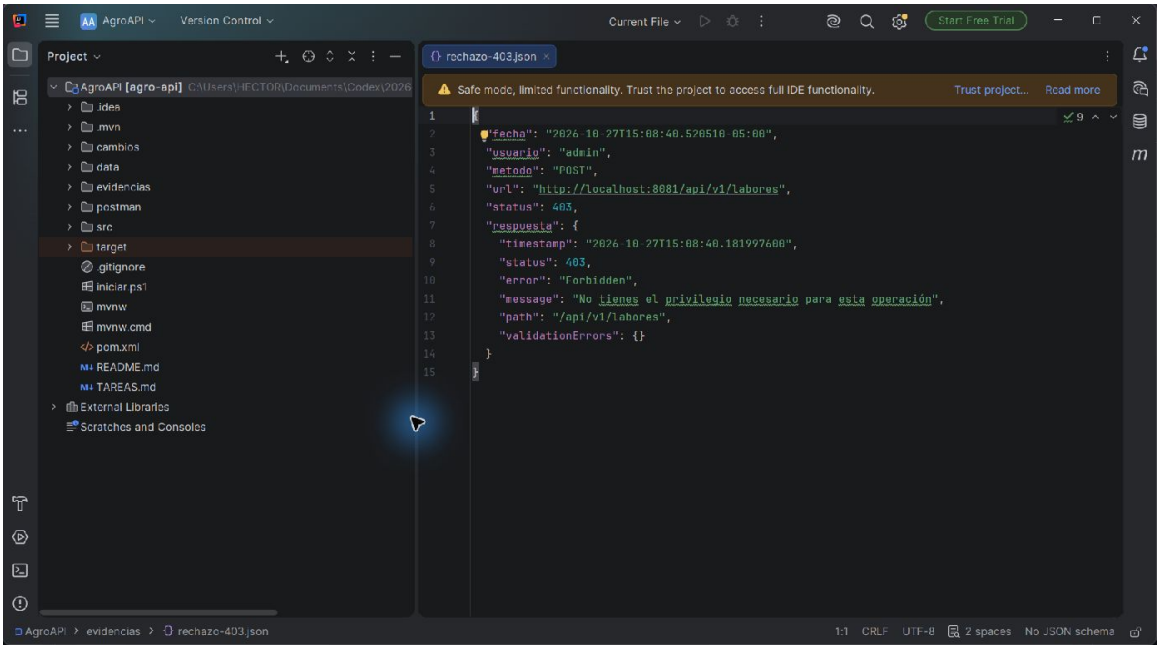
2. Seguridad y reglas

Cuenta / rol	Privilegios y alcance
tecnico1 / TECNICO	Lectura, registro y corrección de labores propias dentro de 24 h.
jefe1 / JEFE_CAMPO	Lectura, registro, corrección global, eliminación y cierre.
admin / ADMINISTRADOR	Lectura y USUARIO_GESTIONAR; no registra labores.
calidad1 / SUPERVISOR_CALIDAD	SC-B: lectura y corrección global; sin crear ni cerrar.

Contraseña semilla común: Agro2026*. Se guarda con BCrypt. El filtro verifica firma y caducidad del JWT y aplica privilegios, sin comparar nombres de rol en controladores. Android cifra la sesión con una clave de Android Keystore.

RN-01: campaña cerrada impide registrar/corregir (409). RN-02: autoría y menos de 24 h para alcance restringido (403). RN-03: el cierre exige cosecha (409). RN-04: fumigación no supera dosis por hectárea por superficie (400 bajo cantidad).

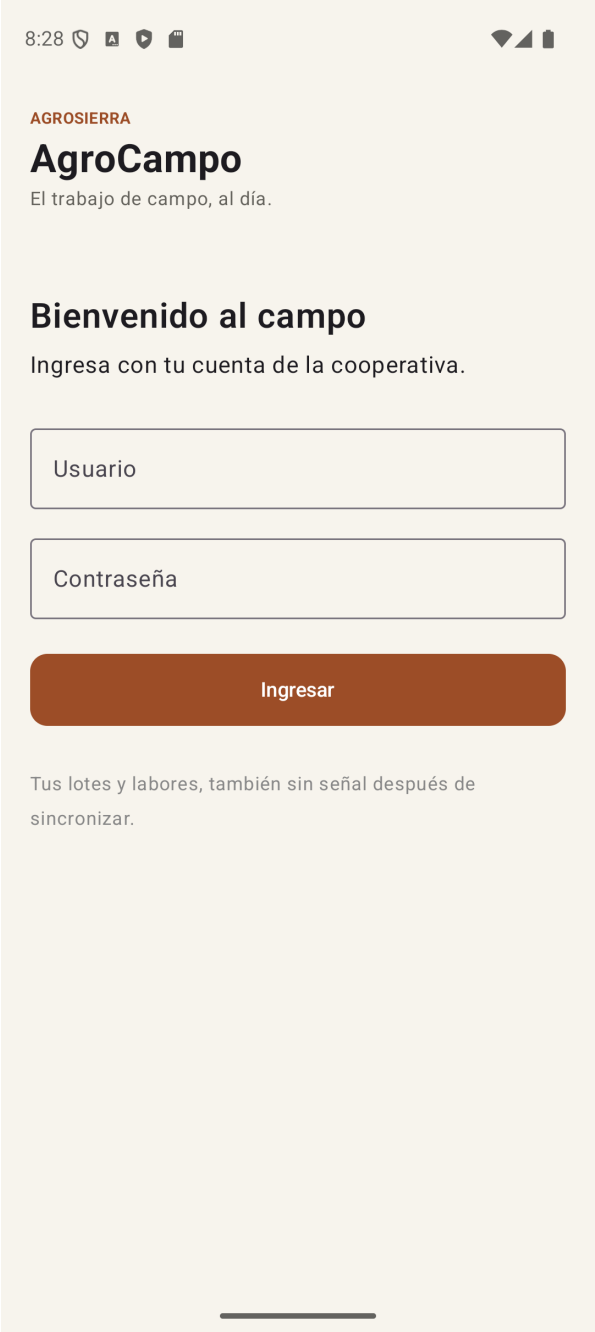
Rechazo real: POST /api/v1/labores con admin devuelve 403. La respuesta archivada se muestra abierta en IntelliJ; GET /usuarios/me sigue respondiendo 200 con esa sesión.




```
HTTP 403 Forbidden | POST /api/v1/labores | usuario: admin
{
  "timestamp": "2026-10-27T15:08:40.181997600",
  "status": 403,
  "error": "Forbidden",
  "message": "No tienes el privilegio necesario para esta operación",
  "path": "/api/v1/labores",
  "validationErrors": {}
}
```

3. Inicio de sesión

Credenciales válidas dan acceso al área protegida. Las incorrectas muestran un mensaje específico y la falta de red se distingue. 401 borra la sesión; 403 la conserva.



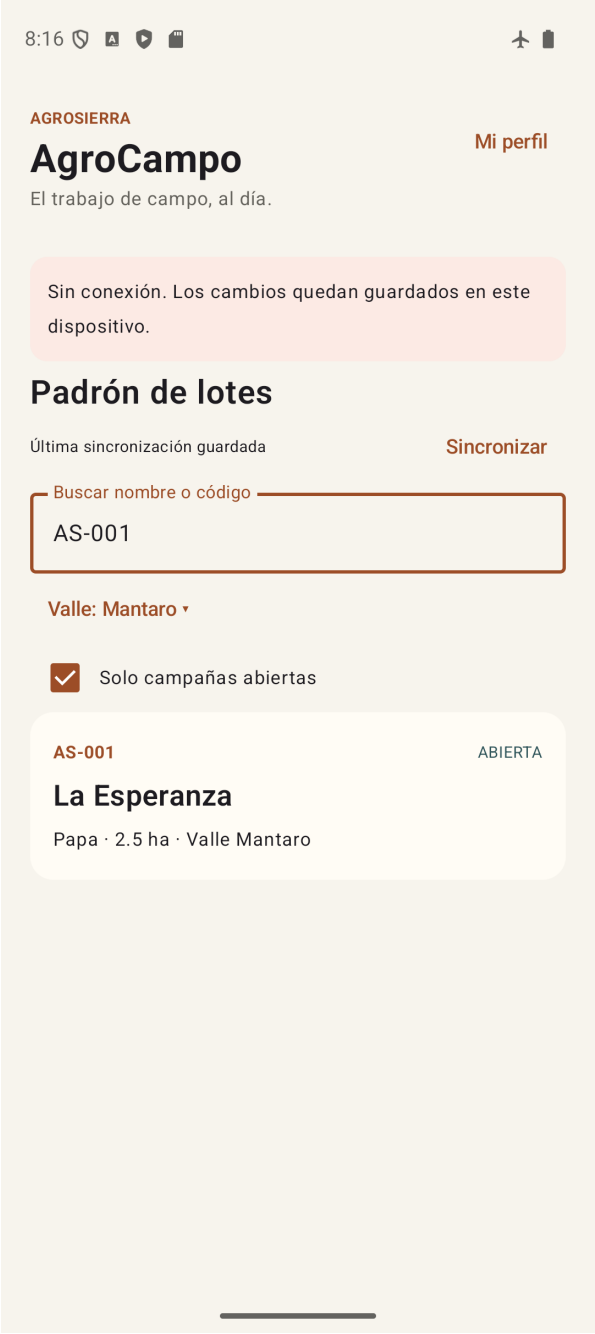
Acceso de AgroCampo.



Mensaje específico de credenciales incorrectas.

4. Consulta y filtros sin red

En modo avión el padrón sale de SQLite. SC-A combina valle, búsqueda y campañas abiertas en la consulta local. SC-D obtiene los pendientes por lote de un COUNT observable.



Mantaro + AS-001 + campañas abiertas, sin cobertura.



Reapertura en frío: registro conservado y contador de una labor pendiente.

5. Pendiente y confirmación

Se registra ExamenOffline-151800: cantidad 2.0 y tres jornales. SQLite asigna -1 y pendiente=1. Al recuperar conexión se envía primero y el servidor confirma el ID 23. La fila se sustituye en una transacción.



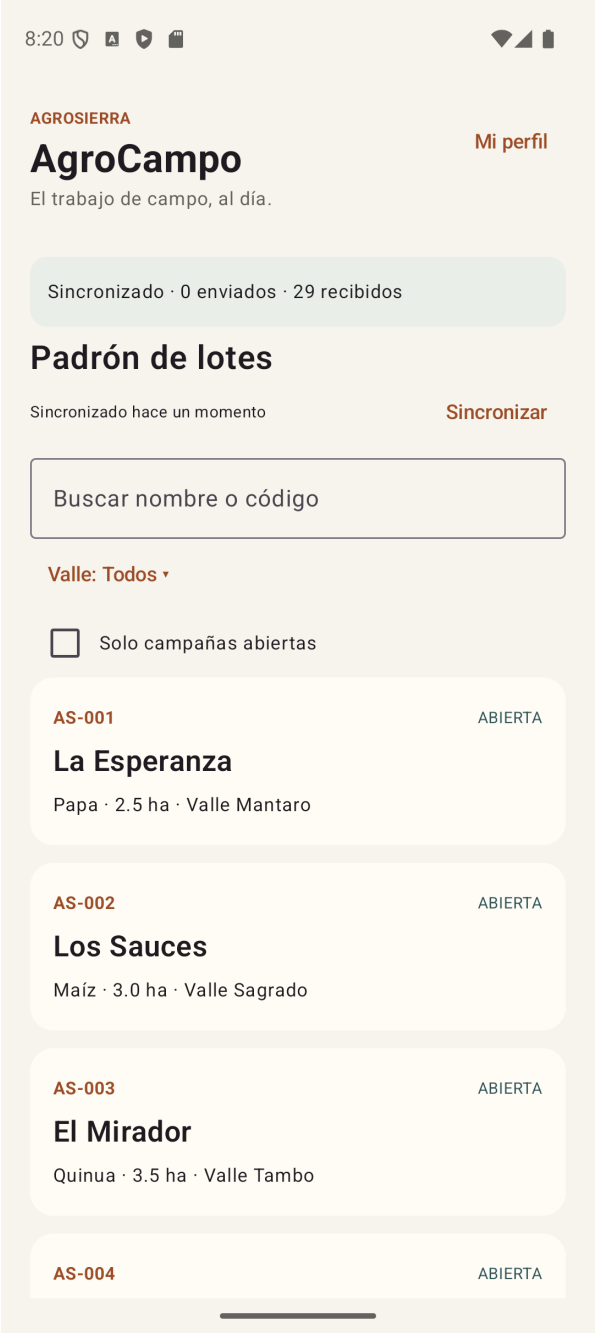
Modo avión, ID -1 y pendiente visibles.



ID definitivo positivo; no se duplica ni desaparece la labor.

6. Sesión persistente y corrección

Un token vigente permite reabrir directamente. Se muestra la antigüedad de la sincronización. La labor se corrige a cantidad 2.75 y cuatro jornales y se verifica en SQLite y por HTTP.



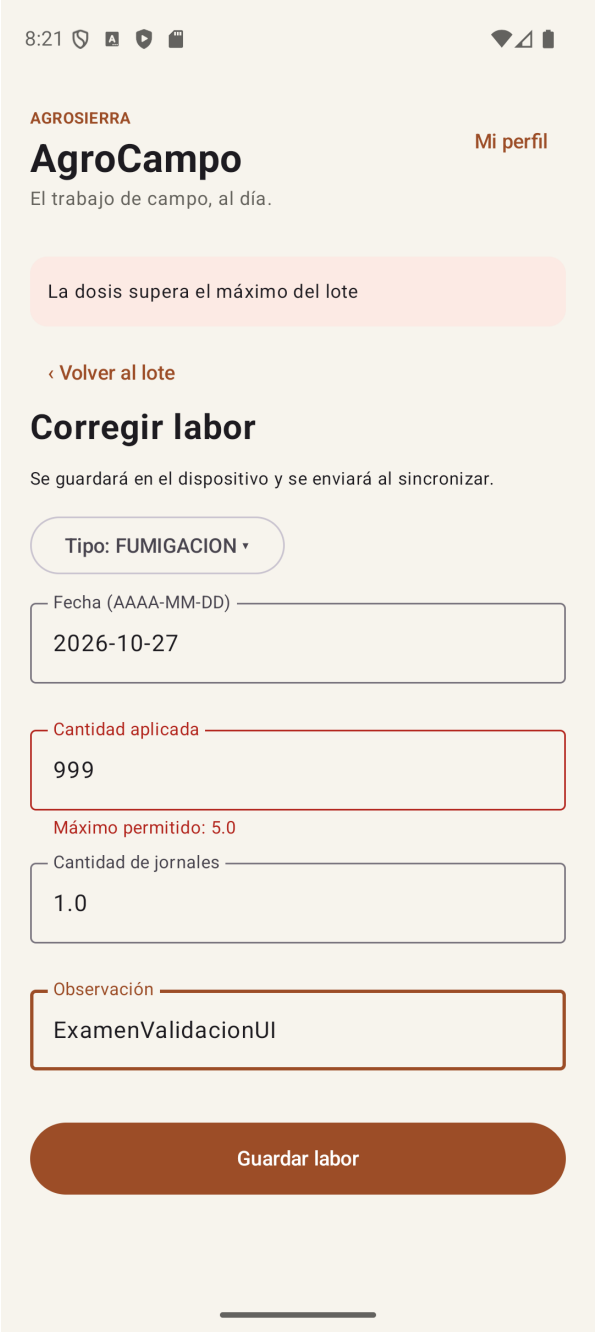
Entrada directa y señal de sincronización.



Corrección confirmada; respuesta en labor-confirmada-servidor.json.

7. Validación y conflicto

Una fumigación con cantidad 999 excede el máximo 5 del lote. AgroAPI responde 400 y el detalle llega al campo cantidad. Se corrige a 1 y se envía. Cerrar Los Sauces sin cosecha devuelve 409, sin cerrar sesión.



RN-04: máximo permitido 5.0 debajo del campo.



RN-03: motivo del conflicto.

8. Eliminación y cierre correcto

jefe1 elimina la labor temporal ExamenValidacionUI. DELETE devuelve 204, no se deserializa el cuerpo y se limpia la cola. Después se cierra La Esperanza porque tiene cosecha. La labor offline demostrada se conserva.



El registro temporal ya no aparece.



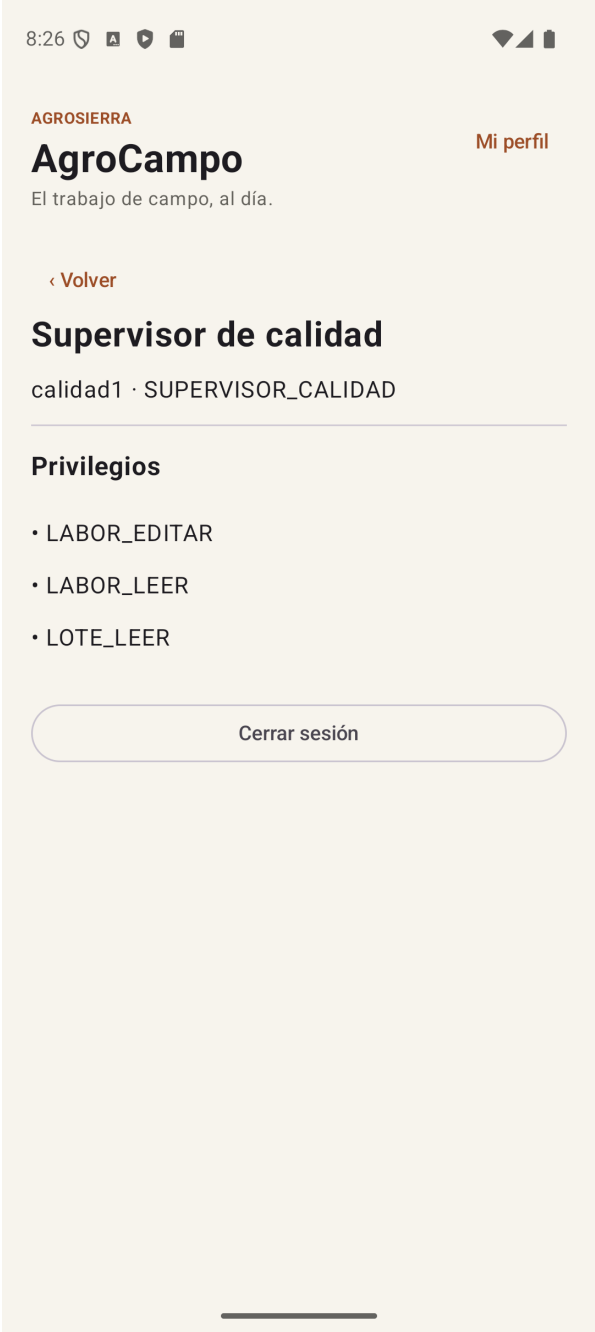
Cierre confirmado; desaparecen Registrar y Corregir.

9. Supervisor de calidad

SC-B agrega lectura y LABOR_EDITAR global en la base de roles. Android recibe privilegios y alcance. La prueba HTTP permite corregir una labor ajena y antigua y rechaza crear/cerrar con 403.



Ofrece Corregir; no Registrar labor ni Cerrar campaña.



Perfil con rol y tres privilegios.

10. Pruebas y resultados

Ámbito	Resultado
AgroAPI / integración HTTP	9 pruebas; 0 fallos, errores u omisiones
AgroCampo / commonTest	13 pruebas; 0 fallos, errores u omisiones
Postman con Newman	27 peticiones y 30 comprobaciones aprobadas; cierre opcional omitido

Cada prueba común utiliza una base SQLite en memoria propia y MockEngine. Se comprueban DTO, errores, registro offline, orden de sincronización, reintento sin duplicados, conservación ante fallo, 204, filtros, supervisor, jornales, migración, contador y credenciales incorrectas. Los fixtures no se usan en producción.

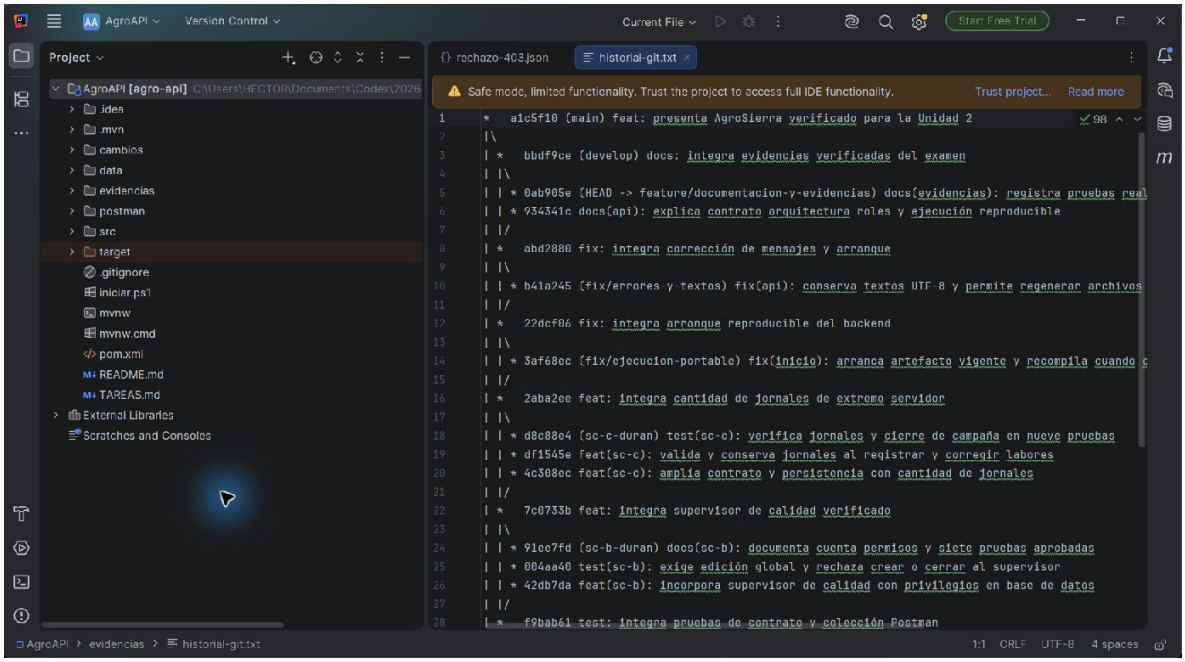
```
AgroTest > contadorProvieneDeLaColaYSeLimpiaAlConfirmar PASSED
AgroTest > enviaAntesDeDescargarYSincronizarDosVecesNoDuplica PASSED
AgroTest > registroSinRedPersisteIdNegativoYEmiteFlow PASSED
AgroTest > supervisorNoPuedeRegistrarAunqueInvoqueElRepositorio PASSED
AgroTest > supervisorRecibePrivilegiosYAlcanceDesdeElContrato PASSED
AgroTest > migracionDeJornalesConservaLaboresPrevias PASSED
AgroTest > falloDeEnvioNoBorraPendienteNiDescarga PASSED
AgroTest > filtroDeCampaniaSeCombinaConValleYBusquedaSinRed PASSED
AgroTest > jornalesRecorrenDtoDominioYBaseLocal PASSED
AgroTest > eliminar204NoIntentaLeerUnCuerpo PASSED
AgroTest > loginIncorrectoNoSeConfundeConSesionVencida PASSED
AgroTest > convierteDtoSinSerializacionEnDominio PASSED
AgroTest > traduceErroresPorCodigoYConservaCampos PASSED
BUILD SUCCESSFUL in 1m 13s
```

La suite HTTP comprueba BCrypt, expiración de una hora, token ausente/alterado/vencido, 403 sin invalidar sesión, paginación, CRUD, idempotencia, RN-01 a RN-04, supervisor, jornales y cierre positivo. XML y registros completos en evidencias/.

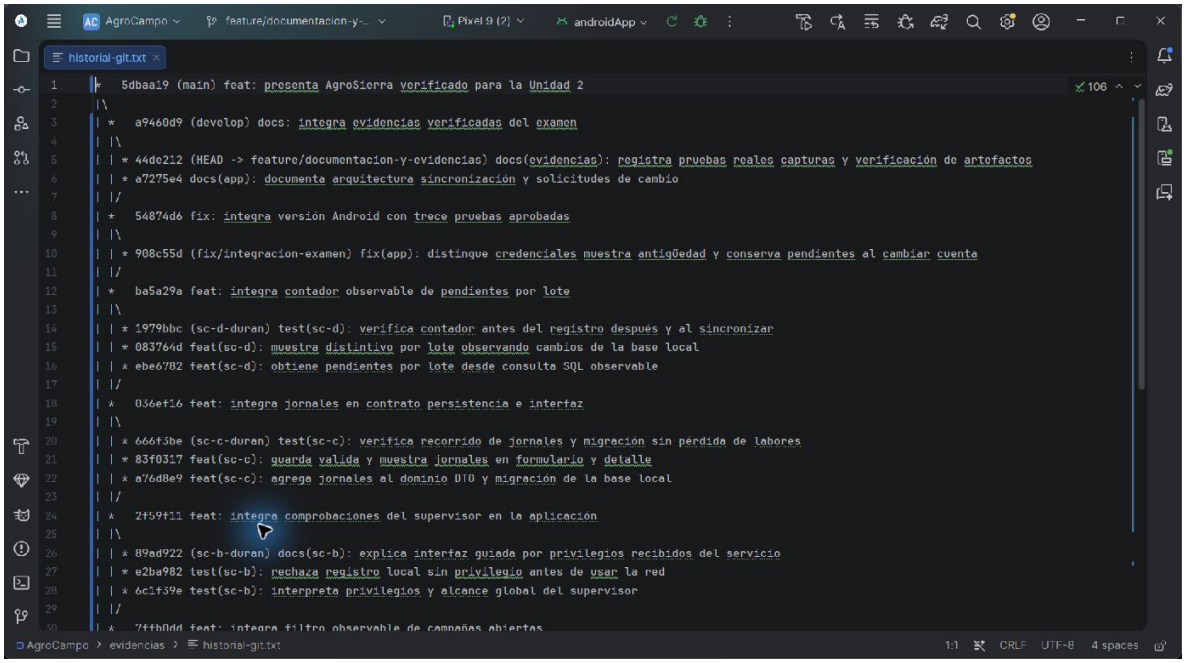
Los SHA-256 del APK y del JAR están en verificacion-final.json. La colección y el entorno de postman/ no contienen JWT reales.

11. Trazabilidad del desarrollo

Las capturas muestran git log --graph --oneline --decorate --all abierto en los IDE. Se conservan ramas feature, fix, sc-a-duran, sc-b-duran, sc-c-duran y sc-d-duran, integración en develop y fusiones hacia main.



AgroAPI: seguridad, contrato, pruebas y cambios de servidor.



AgroCampo: aplicación, pruebas y cuatro solicitudes identificadas.

Estos repositorios se inician realmente el 27 de octubre; no acreditan desarrollo repartido entre fechas anteriores de la unidad. La publicación prepara solicitudes de incorporación reales y v1.0-unidad2, que deben comprobarse en GitHub. No se presentan fusiones locales como solicitudes remotas cerradas.

12. Ejecución y entrega

AgroAPI: abrir el backend en IntelliJ IDEA. iniciar.ps1 compila si hace falta y arranca el puerto 8081 con la base y las semillas. No ejecutar una segunda instancia sobre un puerto ocupado.

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\iniciar.ps1
```

AgroCampo: abrir en Android Studio, sincronizar Gradle, iniciar Pixel 9 (2) y ejecutar androidApp. ApiConfig centraliza 10.0.2.2:8081/api/v1/.

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\verificar.ps1
```

Recorrido breve: entrar como tecnico1, sincronizar, activar modo avión, registrar en un lote abierto, cerrar/reabrir y comprobar pendiente; reconectar y comprobar ID definitivo. Usar jefe1 para eliminar una labor de prueba y los dos casos de cierre; admin en Postman para el 403 de registro.

Archivo o carpeta	Contenido
README.md	Ejecucion, contrato y roles
postman/	Coleccion y entorno de pruebas
evidencias/	Capturas, bitacora, XML y registros
cambios/	Solicitudes SC-A a SC-D
shared/src/commonTest	Pruebas con MockEngine y SQLite en memoria
v1.0-unidad2	Etiqueta del commit evaluado